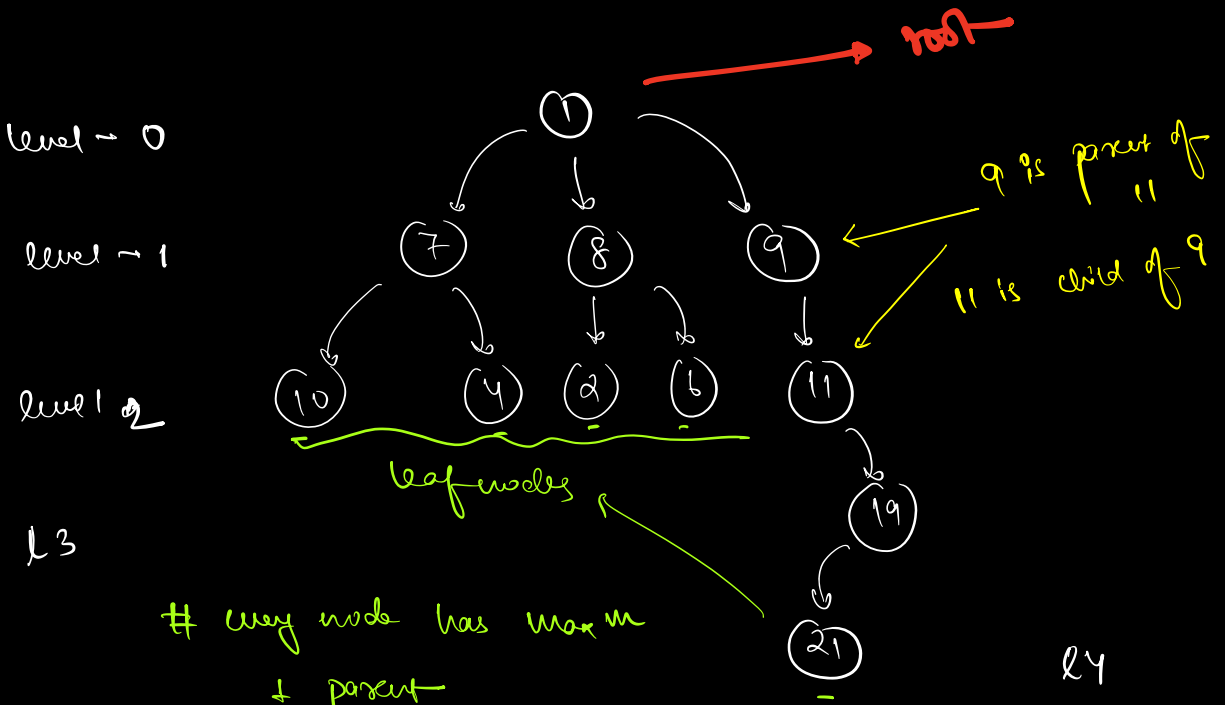


TreeNode

put data

TreeNode C1

" C2
" C3
" /
" }



every node has max m
+ parent

no cycle

11 nodes 10 edges

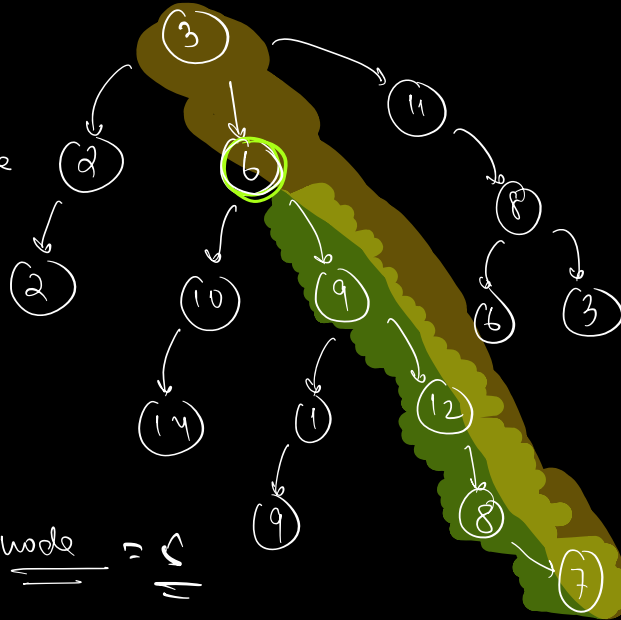
N nodes $\rightarrow$ N-1 edges

∴ Height of a node:

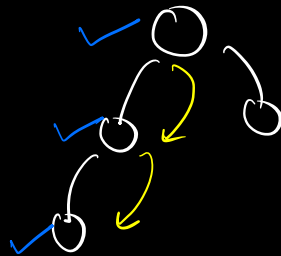
max distance from
that node to leaf node

↓
height of the
tree

→ height of root node = 4



⇒

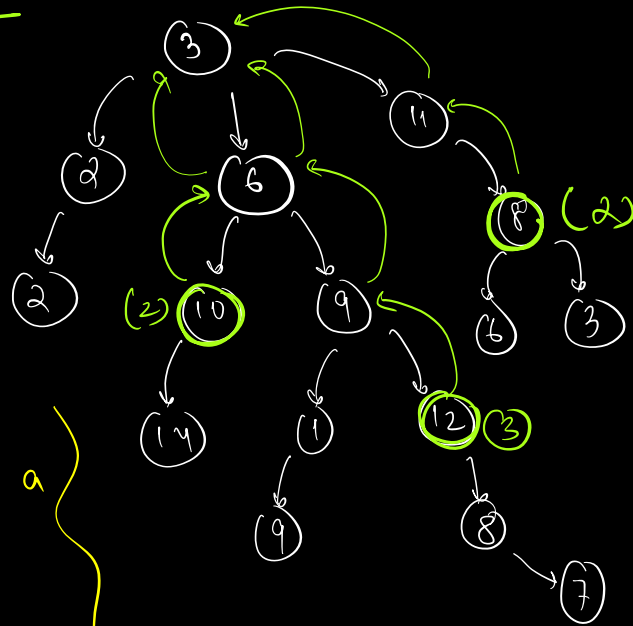


$$h = 2 \mid \underline{3}$$

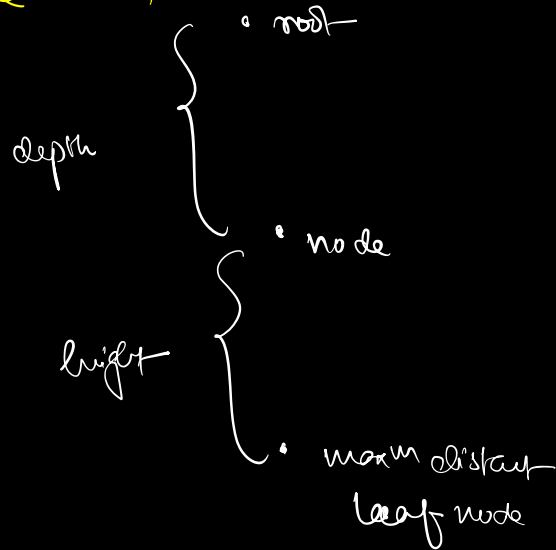
○

$$h = 0 \mid 1$$

⇒ depth of a node



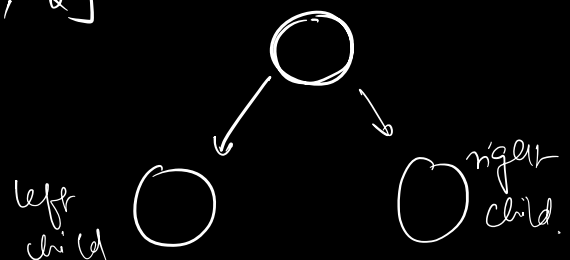
{ There is always a
single path from
root → node }

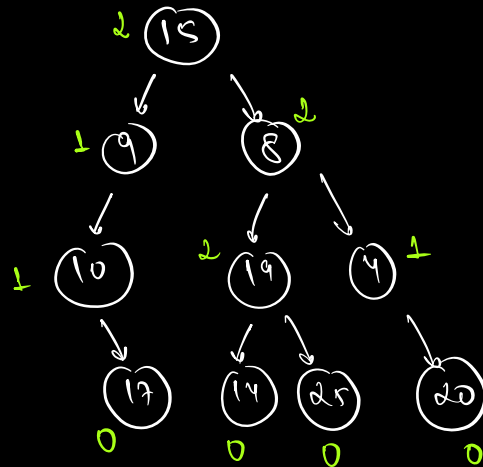


⇒ Binary tree :

each node can have at max
2 children

[0, 1, 2]





```

class PreNode {
    int data;
    PreNode left;
    PreNode right;
}
  
```

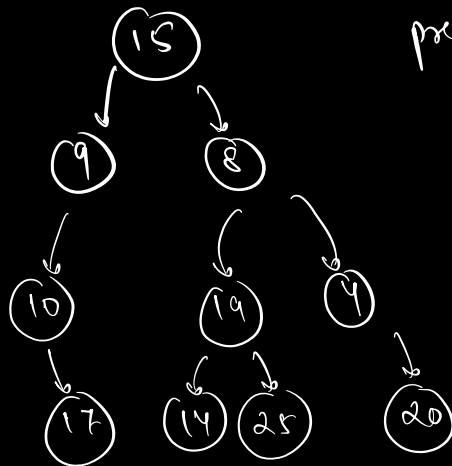
∴ Pre is a recursive data structure.

⇒ Traversals

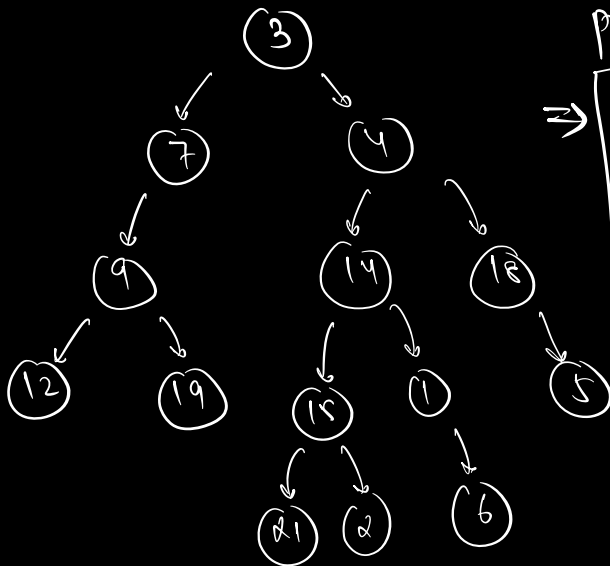
- i) Pre Order
- ii) In Order
- iii) Post Order
- iv) Level Order
- v) Level wise level order

1) Pre Order traversal:-

ROOT LEFT RIGHT



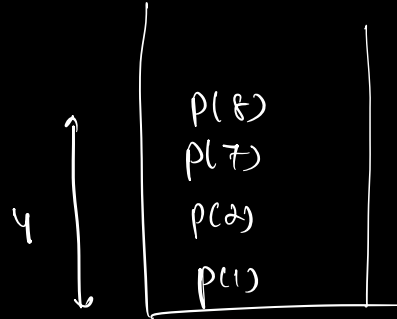
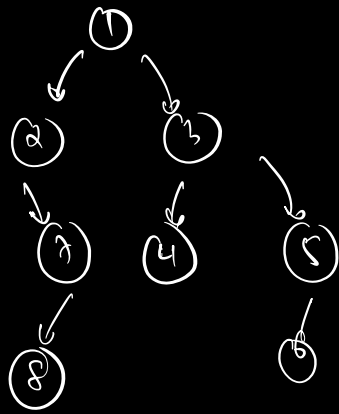
pre $\Rightarrow$ $\begin{bmatrix} 15 & 9 & 10 & 17 & 8 \\ 19 & 14 & 25 & 4 & 20 \end{bmatrix}$



preorder
 $\Rightarrow$ $\begin{bmatrix} 3 & 7 & 9 & 12 & 19 & 4 & 14 \\ 15 & 21 & 2 & 1 & 6 & 18 & 5 \end{bmatrix}$

```

preOrder ( root ) {
    if ( root == null ) {
        return
    }
    print ( root.data )
    preOrder ( root.left )
    preOrder ( root.right )
}
  
```



$TC \Rightarrow O(N)$
 $SC \Rightarrow O(\text{height of tree})$

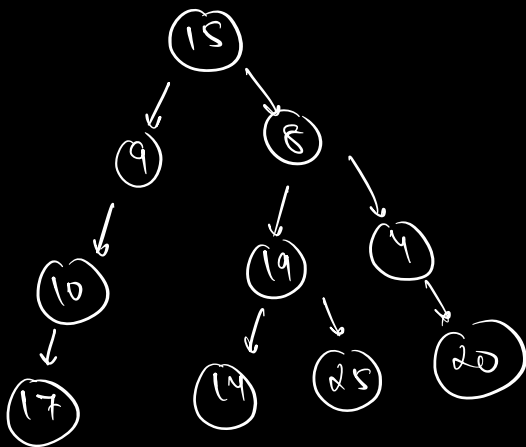
[1 2 7 8 3 4 5 6]

$\Rightarrow$ cc

↓ No | Doubt

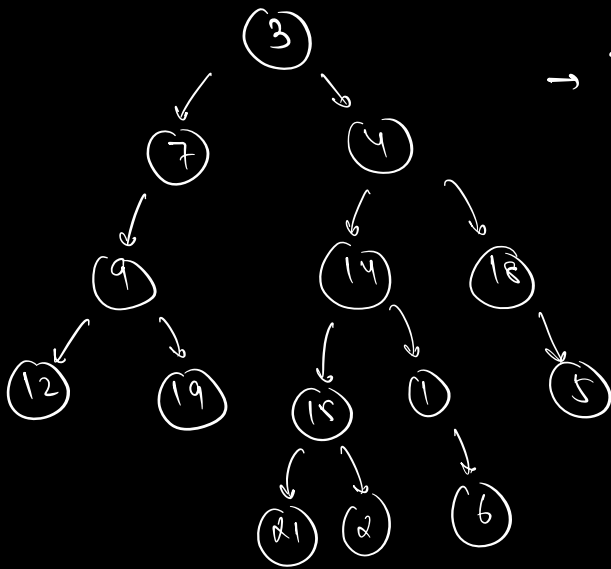
11) Inorder

left ROOT right



$\rightarrow$

17	10	9	15	14
19	25	8	4	20



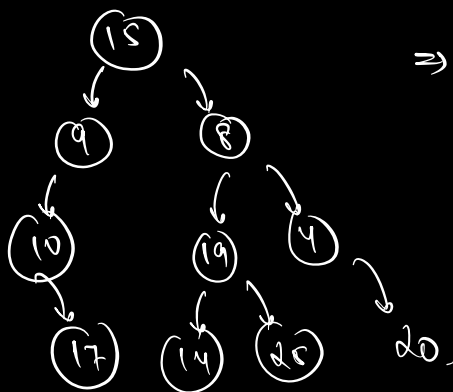
→ [12 9 19 7 3 21 15 2
14 1 6 4 18 5]

```

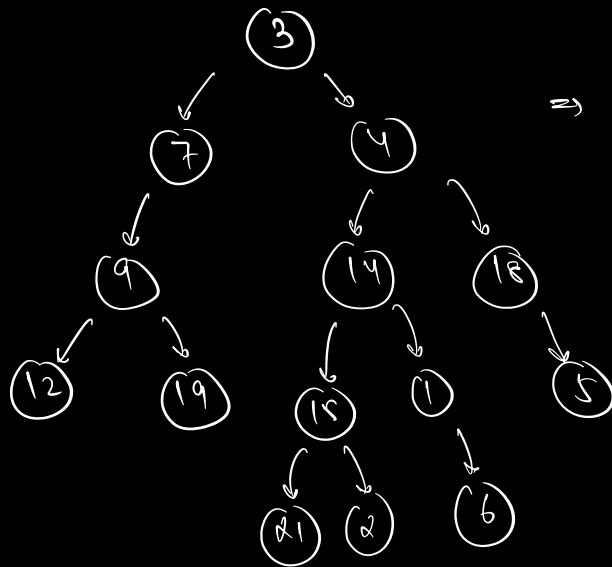
inOrder ( root ) {
    if ( root == null ) {
        return
    }
    inOrder ( root->left )
    print ( root->data )
    inOrder ( root->right )
}
  
```

⇒ post Order traversal:

left right ROOT



⇒ post → [17 10 9 14 20
19 20 4 8 15]



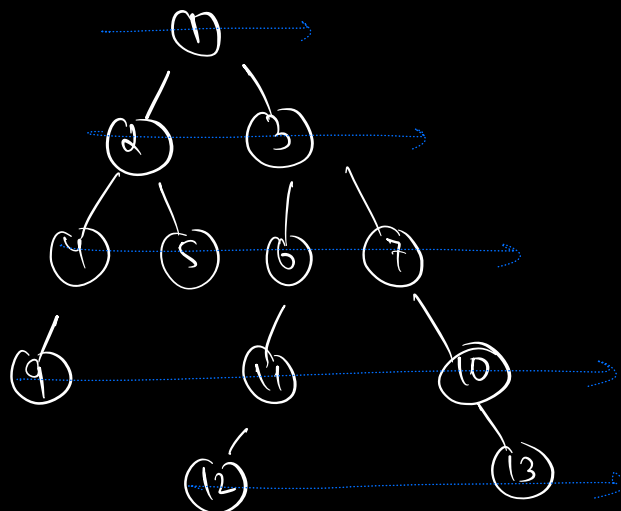
$\Rightarrow \begin{bmatrix} 12 & 19 & 9 & 7 & 21 & 2 & 15 \\ 6 & 1 & 14 & 5 & 18 & 4 & 3 \end{bmatrix}$

```

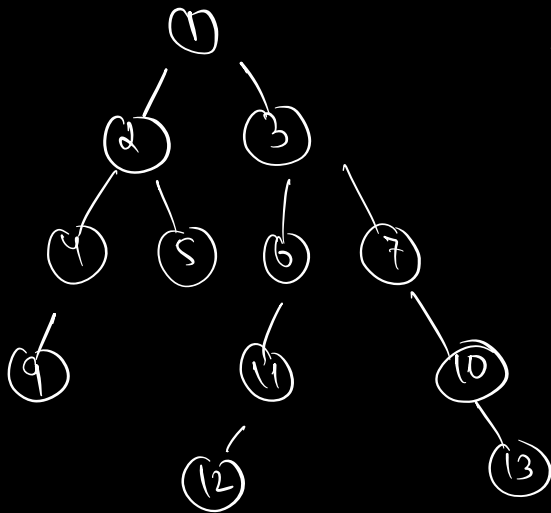
postOrder ( root ) {
    if ( root == null ) {
        return
    }
    postOrder ( root.left )
    postOrder ( root.right )
    print ( root.data )
}

```

→ level order



o/p $\Rightarrow$ 1 2 3 4 5 6 7 9 11 10 12 13



~~1 2 3 4 5 6 7 9 11 10 12 13~~
12

o/p $\Rightarrow$ 1 2 3 4 5 6 7 9 11 10 12 13

queue < TreeNode > q

q.enqueue(root)

while (!q.isEmpty()) {

 x = q.dequeue();

 print(x.data);

 if (x.left != null)

 q.enqueue(x.left)

 if (x.right != null)

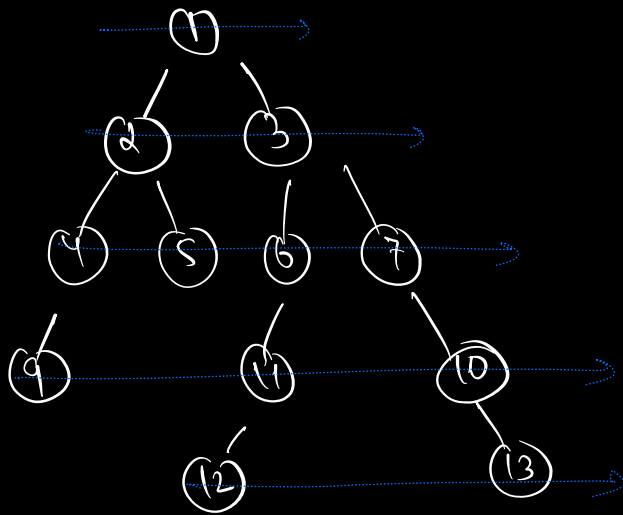
 q.enqueue(x.right)

}

$T.C \Rightarrow O(N)$

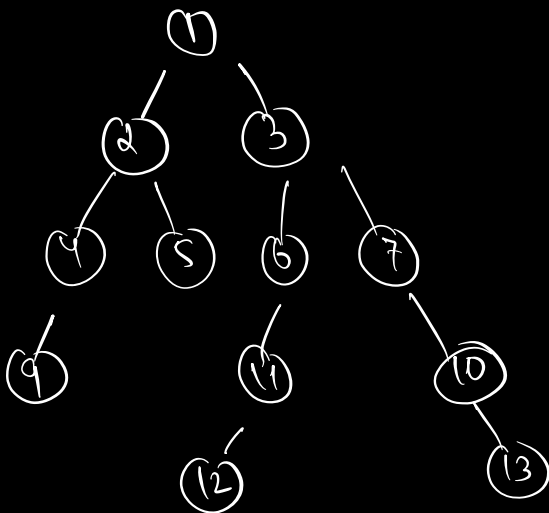
$S.C \Rightarrow O(\text{width of tree})$

⇒ level wise level order:



O/P ⇒

- 1
- 2 3
- 4 5 6 7
- 9 11 10
- 12 13



~~NULL~~

~~1 2 3 4 5 6 7 8 9 10~~
~~12 13~~

→ 1
 → 2 3
 → 4 5 6 7
 → 9 11 10
 → 12 13

q →

q.enqueue(root)

q.enqueue(NULL)

while (q.size() > 1) {

u = q.dequeue;

if (u == NULL) {

print(u)

q.enqueue(NULL)

} else {

print(u)

if (u.left != null)

q.enqueue(u.left)

if (u.right != null) {

q.enqueue(u.right)

}

TC → {
SC → { same as
last

Recursion → Recursion + 22 (42)